

TraceLight.

Демо апликација за скицирање на фотографии

Драгана Усовиќ · 221043

ментор: проф. д-р Ивица Димитровски

Дигитално процесирање на слика · ФИНКИ

02 • ВОВЕД

Што е скицирање на фотографии?

Дефиниција

Трансформација на фотографија во цртеж што имитира работа со молив или пенкало на хартија. Се добива со комбинирање на класични оператори на дигитално процесирање на слика, без машинско учење.

Зошто е значајно за ДПС?

- Демонстрира како основните оператори се комбинираат во еден сложен ефект
- Користи градиенти, конволуција, мешање на канали и квантизација
- Идеална тема за визуелизација на меѓучекори (pipeline view)

Демо апликација TraceLight

Веб-апликација која ги обединува трите имплементирани алгоритми во еден интерфејс. Корисникот прикачува слика, избира ефект и параметри, и веднаш ги гледа резултатот и меѓучекорите.

01

Молив-скица

Color dodge техника — грејскејл, инверзија, Гаусов филтер, мешање.

02

Детекција на рабови

Три класични алгоритми: Sobel, Laplacian, Canny.

03

Дирекционо штрафирање

Линии распоредени според градиентната насока (со Sobel).

Стек на технологии

- **Python 3.10+** Имплементациски јазик за сите алгоритми и UI
- **OpenCV** `cv2.imdecode`, `cv2.cvtColor`, `cv2.GaussianBlur`, `cv2.Sobel`, `cv2.Canny` ...
- **NumPy** Векторизирана аритметика, маски, `float32` операции
- **Streamlit** Веб-интерфејс: прикачување, слајдери, табови, преземање
- **Matplotlib** Визуелизација на меѓучекори и хистограми на интензитет

05 · ОСНОВИ

Дигитална слика како матрица

Слика = 3-димензионална матрица од пиксели: $H \times W \times C$
H = висина, W = ширина, C = број канали (3 за RGB, 1 за грејскејл)

Важно: OpenCV користи BGR (не RGB)

Историско наследство од Windows BMP форматот. При прикажување во Streamlit (RGB), сликата мора да се конвертира со `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

Вчитување на слика

```
file_bytes = np.frombuffer(
    uploaded.read(), np.uint8)
img = cv2.imdecode(
    file_bytes, cv2.IMREAD_COLOR)
```

05 · ОСНОВИ

Конверзија во грејскејл

Формула (ITU-R BT.601)

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

Зошто овие бројки?

- Соодветствуваат на човечката перцепција
- Зелена компонента е најосетлива, сината најмалку
- Едноставниот просек $(R+G+B)/3$ дава помрачна, помалку реална слика

OpenCV повик

```
gray = cv2.cvtColor(img,  
                    cv2.COLOR_BGR2GRAY)
```

05 · ОСНОВИ

Гаусов филтер

Линеарен филтер за смазнување. Централниот пиксел има најголема тежина, тежините опаѓаат со растојание според Гаусова распределба.

$$G(x, y) = (1 / 2\pi\sigma^2) \cdot \exp(- (x^2 + y^2) / 2\sigma^2)$$

Параметри

- Големина на јадрото $k \times k$ (непарен број)
- σ — стандардна девијација (контролира острина)
- Поголем $k \rightarrow$ помеки и пошироки потези во резултатот

OpenCV повик

```
cv2.GaussianBlur(img, (k, k), 0)
```

Со 0 за σ , OpenCV автоматски пресметува: $\sigma = 0.3 \cdot ((k-1)/2 - 1) + 0.8$

Преглед на алгоритам

Color Dodge техника — истата што се користи во Photoshop / GIMP. Имитира темни молив-линии на бела хартија.



Излез: едноканална uint8 слика (бела хартија со темни молив-линии)

06 · МОЛИВ-СКИЦА

Чекори 1 и 2 — грејскејл и инверзија

ЧЕКОР 01

Грејскејл конверзија

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

Каналите се сведуваат на еден тонален канал. Сите следни операции работат на оваа едноканална слика.

ЧЕКОР 02

Инверзија

$$I_{inv}(x, y) = 255 - I(x, y)$$

Фотографски негатив. Темните области стануваат светли. Овој чекор е клучен — без него color dodge би осветлил место да затемни.

Чекори 3 и 4 — Гаусов филтер и color dodge

ЧЕКОР 03

Гаусов филтер на инвертираната слика

$$B = \text{GaussianBlur}(I_{\text{inv}}, (k, k), 0)$$

Темните инверзни области се разлеваат.

ЧЕКОР 04

Color Dodge мешање

$$S = \min(G \cdot 255 / (255 - B + \epsilon), 255)$$

- $B \approx 255 \rightarrow$ резултат се режи на бела (хартија)
- B мала $\rightarrow$ оригиналната грејскејл вредност
- $\epsilon = 1e-6$ — спречува делење со нула

06 · МОЛИВ-СКИЦА

Резултат и опционо мешање

Карактеристики на излезот

- Едноканална uint8 слика — бела хартија со темни молив-линии
- Бимодална дистрибуција на интензитет: големи скокови кај 0 (линии) и 255 (хартија)
- Темните потези соодветствуваат на рабови во оригиналот

Опционо мешање со интензитет

$$Out = \alpha \cdot S + (1 - \alpha) \cdot gray \quad \text{каде } \alpha \in [0.1, 1.0]$$

07 · РАБОВИ

Детекција на рабови

Што е раб?

Локација каде интензитетот нагло се менува, односно границата помеѓу две области со различна осветленост. Рабовите се основа на секој цртеж со линии.

Прв извод

$$\nabla I = (\partial I / \partial x, \partial I / \partial y)$$

Бараме голема магнитуда $\|\nabla I\|$. Оператори: Sobel, Prewitt, Roberts.

Втор извод

$$\nabla^2 I = \partial^2 I / \partial x^2 + \partial^2 I / \partial y^2$$

Бараме нула-пресеци или голема апсолутна вредност. Оператори: Laplacian, LoG.

07 · РАБОВИ

Sobel оператор

Прв-изводен оператор. Со две 3×3 јадра ги пресметува хоризонталниот и вертикалниот градиент.

Sx (хоризонтално)

-1	0	+1
-2	0	+2
-1	0	+1

Sy (вертикално)

-1	-2	-1
0	0	0
+1	+2	+1

Магнитуда на градиент

$$\begin{aligned}G_x &= S_x * I \\G_y &= S_y * I \\||\nabla I|| &= \sqrt{G_x^2 + G_y^2}\end{aligned}$$

```
cv2.Sobel(img, cv2.CV_64F, dx, dy, ksize)
```

07 • РАБОВИ

Laplacian оператор

Втор-изводен изотропен оператор. Не зависи од насока — детектира рабови во сите ориентации со едно јадро.

Дискретно јадро 3×3

0	-1	0
-1	4	-1
0	-1	0

Формула

$$\nabla^2 I = \partial^2 I / \partial x^2 + \partial^2 I / \partial y^2$$

- Изотропен — еднаков одговор за рабови во која било насока
- Многу чувствителен на шум — задолжително замазнување
- `cv2.Laplacian(img, cv2.CV_64F, ksize) + cv2.convertScaleAbs`

Canny алгоритам

Повеќестепенски алгоритам, еден од најкористените во практика. John Canny, 1986.

- 1 Гаусово смазнување
- 2 Sobel градиенти — магнитуда и насока
- 3 Non-maximum suppression — истенчување на рабовите
- 4 Двојно прагирање (low / high)
- 5 Hysteresis — се чуваат слаби рабови поврзани со силни

Препорачано: $\text{low} \approx \text{high} / 2$ • `cv2.Canny(img, low, high)`

08 · ШТРАФИРАЊЕ

Дирекционо штрафирање

Уметничкото правило

Уметникот ги црта линиите вдолж работ, не препреку него. Sobel ни ја дава не само магнитудата на градиентот, туку и неговата насока.

Градиентна насока

$$\theta = \text{atan2}(G_y, G_x) \bmod 180^\circ$$

Насока на работ

$$\varphi = (\theta + 90^\circ) \bmod 180^\circ$$

$\bmod 180^\circ$ — затоа што линија е двонасочна (ϑ и $\vartheta + 180^\circ$ се иста ориентација)

Алгоритам за дирекционо штрафирање

- 1 Грејскејл + Гаусово смазнување
- 2 Sobel градиенти G_x , G_y
- 3 Поларна форма: $\|\nabla I\|$ и θ
- 4 Насока на работ: $\phi = (\theta + 90^\circ) \bmod 180^\circ$
- 5 Квантизација на ϕ во N корпи (4 или 8)
- 6 Создавање на N паралелни линиски текстури
- 7 Edge mask: $\|\nabla I\|_{\text{norm}} > \text{threshold}$
- 8 Композиција: за секој edge-пиксел, избери текстура според корпата

Резултат: бела хартија со паралелни молив штрафирања ориентирани вдолж рабовите.

Преглед на демо апликацијата

ЛЕВА ЛЕНТА

Контроли

Прикачување на слика, избор на ефект, динамички параметри во зависност од избраниот алгоритам.

ТАБ — RESULT

Резултат

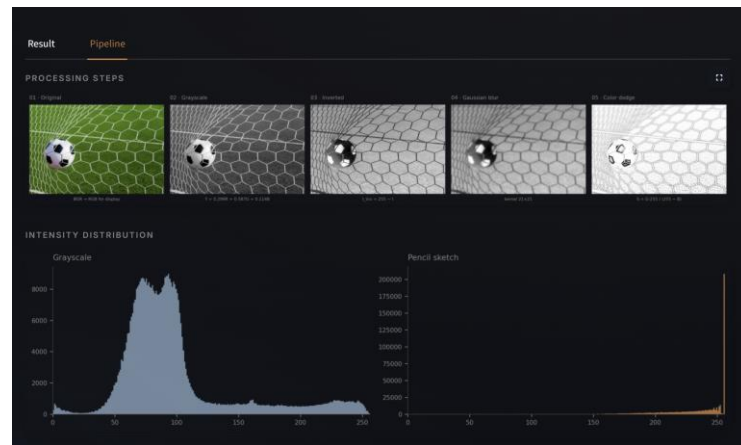
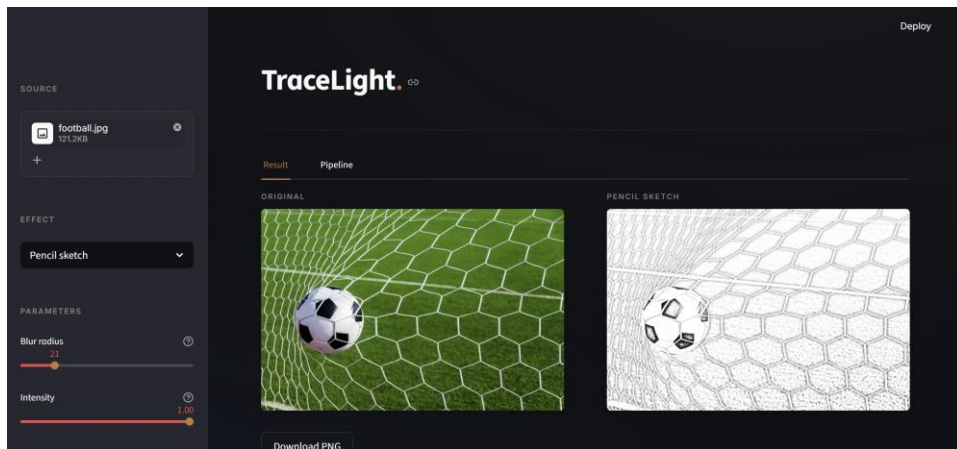
Оригинал и излез страна до страна, со копче за преземање на резултатот како PNG.

ТАБ — PIPELINE

Меѓучекори

Сите интерни чекори се прикажани со matplotlib subplots, заедно со хистограми на интензитет.

Преглед на демо апликацијата



ЗАКЛУЧОК

Скицирање слики

Молив-скица, детекција на рабови и дирекционо штрафирање, сите имплементирани врз основа на истиот сет на класични оператори: грејскејл, Гаусов филтер, Sobel и мешање. Pipeline-визуелизацијата ги прикажува меѓучекорите за подобро разбирање на секој алгоритам.

РЕФЕРЕНЦИ

Gonzalez R. C., Woods R. E. · Digital Image Processing (4th ed.), Pearson, 2018

Canny J. · A Computational Approach to Edge Detection. IEEE PAMI, 1986

Sobel I. · An Isotropic 3×3 Image Gradient Operator. SAIL, 1968

OpenCV documentation · docs.opencv.org



GitHub repository